Pyramid Backstage — Technical Design

Challenge: AADF × Pyramid of Tirana — turn an event *request* into an event-ready *operational plan*, replacing the email / Excel / phone-call chaos.

Team: 2 people, 2 microservices, one shared contract.

- **Elis —** `ops-core` (TypeScript · Node.js · PostgreSQL · NATS): owns all state and deterministic logic.
- **Alvin —** `ai-orchestrator` (Python · FastAPI · LangGraph · Claude · ChromaDB · Redis): owns all intelligence.

The two services talk **only over the payload contract in Section 4**. Neither imports the other's code. Lock that contract early and you can build in total isolation until integration.

---

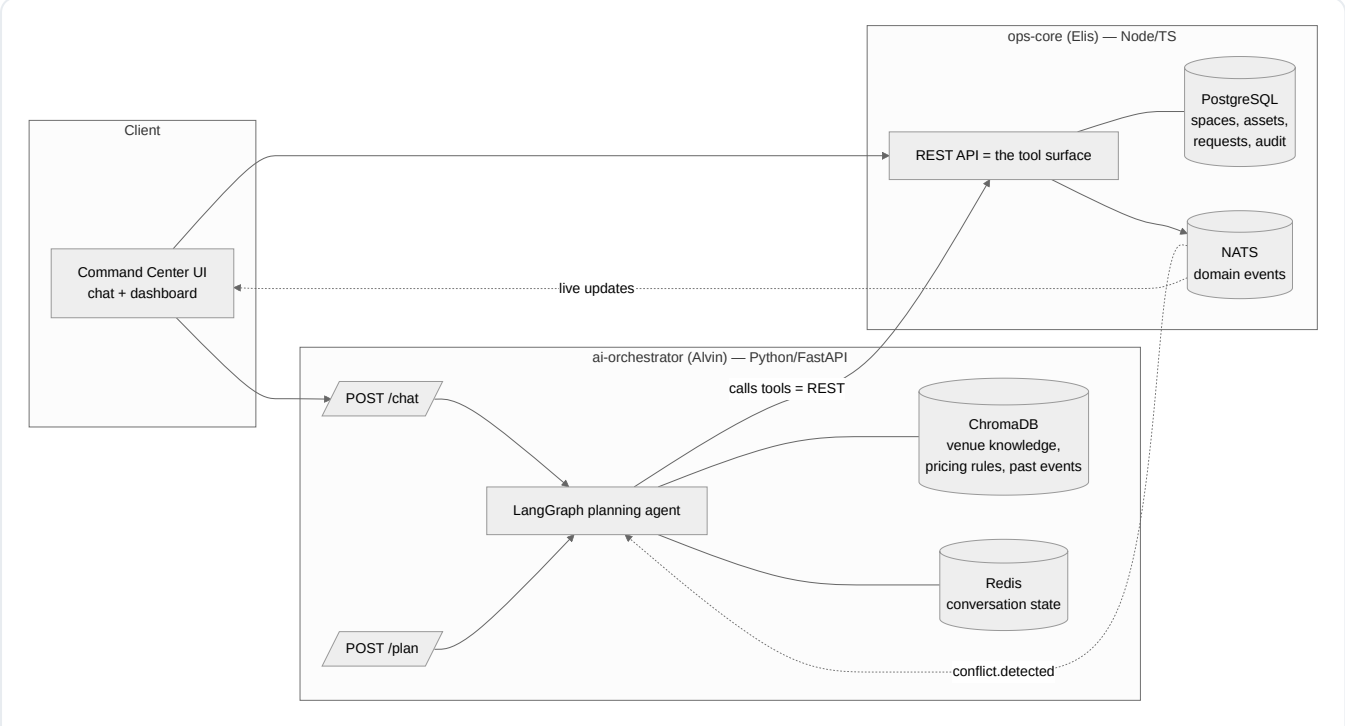
1. The one-sentence system

> *A venue ops team types a messy request — "startup conference, 180 people, late next month, needs a stage and mics" — and the system answers **"yes, here's how"**: matched space, generated quote, reserved assets, conflict check, and a setup/teardown task plan — or **"not as-is, here are the alternatives."***

`ai-orchestrator` is the brain that understands the request and decides what to do. `ops-core` is the system of record that knows what's true (spaces, inventory, schedules, money) and enforces it. The brain never holds domain state; the record never reasons. That split is what lets you work separately.

---

2. Architecture



Why this division wins the challenge: every requirement in the brief is either *state* (Elis) or *reasoning over state* (Alvin), and the headline demo — natural-language request → full operational plan — is literally an agent calling Elis's API as tools. It puts both your strengths on the critical path at the same time.

---

3. Who owns what

Elis — `ops-core`

The deterministic source of truth. No AI. Build real endpoints, not stubs.

- **Domain & schema:** spaces, assets, event requests, reservations, quotes, tasks, audit log.
- **Spaces:** the four floor-0/-1 rooms (Blue, Orange, Green, Yellow) + transitional/entrance/corridor areas. Each has capacity, floor, features, day rate.
- **Inventory engine:** assets (chairs, tables, microphones, screens, projectors, stage units…) with quantity, current location, status. Knows how many are free for any window.
- **Availability engine:** is space X free for `[start, end]`? are N of asset Y free for that window?
- **Reservation engine:** hold/confirm a space + assets for a request; decrement availability atomically.
- **Conflict detection:** double-booked space, over-allocated asset, overlapping setup windows. Returned both on a dedicated check **and** as a `409` when a reservation would violate it.
- **Quotation engine:** pricing rules → line items + total, in **ALL (Albanian Lek)**, i18n-ready.
- **Approval & audit:** immutable log of every decision/change. Status transitions.
- **Event bus (NATS):** emits domain events (Section 5) for the real-time command center and for the AI's proactive conflict explanations.
- **Owns the Command Center UI** (he lists React) — chat panel + live inventory/schedule dashboard. Keep it lean; it's the demo surface, not a product.

Alvin — `ai-orchestrator`

All intelligence. Holds *no* domain state — only conversation context in Redis.

- **NL intake:** free text → structured `EventRequestDraft` (attendees, dates, type, requirements).
- **Planning agent (LangGraph):** orchestrates ops-core tool calls — match space → check availability → check inventory → reserve → quote → generate tasks → detect conflicts → assemble plan. Branches to alternative-proposal on conflict.
- **Conversational copilot:** answers "can we make this happen?", explains conflicts in plain language, recommends next actions and alternatives ("Blue is taken that week; Orange seats 180 in theater layout — shall I hold it?").
- **Proposal generation:** turns the structured plan into a human-readable proposal.
- **RAG (ChromaDB):** venue facts, setup templates, pricing logic, similar past events → grounds recommendations and task generation.
- **Task generation:** reasons out the setup/teardown list from event context, then **persists it through ops-core** so state stays single-sourced.

---

4. The contract (this is the only thing you share)

`ai-orchestrator` treats each `ops-core` endpoint as a LangGraph **tool**. Lock these payload shapes in hour 0; after that, additive-only changes. Put the OpenAPI spec in the repo as the single source of truth.

Base URL is one env var on the AI side: `OPS_CORE_URL`. That's the entire coupling.

4.1 Shared domain objects

```
// Space
{ "id": "space_blue", "name": "Blue Hall", "floor": 0, "capacity": 220,
  "features": ["stage", "av_builtin", "step_free"], "dayRate": 80000, "currency": "ALL" }

// Asset
{ "id": "asset_chair_std", "name": "Standard chair", "type": "seating",
  "totalQuantity": 400, "location": "Storage -1", "status": "active" }

// EventRequest
{ "id": "req_8x2", "title": "FinTech Startup Conference",
  "organizerName": "Acme", "contactEmail": "x@acme.al",
  "expectedAttendees": 180, "eventType": "conference",
  "preferredDates": [{ "start": "2026-07-22T09:00:00Z", "end": "2026-07-22T18:00:00Z" }],
  "requirements": { "layout": "theater", "avNeeded": true, "cateringNeeded": false, "notes":
"needs a stage" },
  "status": "DRAFT" }
// status lifecycle: DRAFT → PROPOSED → APPROVED → SCHEDULED → COMPLETED | REJECTED

// Reservation
{ "id": "resv_1", "requestId": "req_8x2", "spaceId": "space_blue",
  "dateRange": { "start": "...", "end": "..." },
  "assets": [{ "assetId": "asset_chair_std", "quantity": 180 }],
  "status": "HELD" }   // HELD → CONFIRMED | RELEASED

// Quote
{ "id": "quote_1", "requestId": "req_8x2", "currency": "ALL",
  "lineItems": [{ "label": "Blue Hall (1 day)", "qty": 1, "unitPrice": 80000, "subtotal": 80000
}],
  "total": 134000 }

// Task
{ "id": "task_1", "requestId": "req_8x2", "title": "Set up theater seating (180)",
  "phase": "SETUP", "owner": "ops_team", "dueOffsetHours": -4, "status": "todo" }
// phase: SETUP | TEARDOWN

// Conflict
{ "type": "space_double_booked", "spaceId": "space_blue",
  "conflictingRequestIds": ["req_5a1"],
  "window": { "start": "...", "end": "..." },
  "detail": "Blue Hall already reserved for req_5a1 in this window." }
// type: space_double_booked | asset_overallocated | setup_window_overlap
```

4.2 The tool surface (ops-core REST → AI tools)

Tool / Endpoint	Purpose	Returns
`POST /requests`	Create a structured event request	`EventRequest`
`GET /requests/:id`	Full aggregate (request+reservation+quote+tasks+audit)	aggregate
`GET /spaces?minCapacity=&start=&end=`	Match + filter spaces, with availability for the window	`Space[]` (each `+available`)
`GET /spaces/:id/availability?start=&end=`	Check one space	`{ spaceId, available, conflictingRequestIds }`
`GET /assets?type=&quantity=&start=&end=`	Inventory availability for a window	`Asset[]` (each `+availableQuantity`)
`POST /reservations`	Hold space + assets	`Reservation` **or** `409 { conflicts: Conflict[] }`
`POST /reservations/:id/confirm`	Confirm a held reservation	`Reservation`
`POST /quotes`	Generate quote from a request/reservation	`Quote`
`GET /conflicts?spaceId=&start=&end=`	Proactive conflict check	`Conflict[]`
`POST /requests/:id/tasks`	Persist AI-generated task list	`Task[]`
`POST /requests/:id/approve`	Approve → confirm reservations → emit events → audit	`EventRequest`
`POST /requests/:id/reject`	Reject + reason	`EventRequest`
`GET /audit?requestId=`	Decision/change history	`AuditEntry[]`

Standard error contract (so the agent can branch deterministically):

```
// 409 on reservation conflict
{ "error": "conflict", "conflicts": [ /* Conflict[] */ ] }
// 422 validation
{ "error": "validation", "fields": { "expectedAttendees": "required" } }
```

4.3 AI-orchestrator endpoints (what the UI / demo calls)

```
// POST /chat  — conversational copilot (stateful via sessionId)
// in:
{ "sessionId": "s1", "message": "startup conf, 180 ppl, late next month, needs a stage" }
// out:
{ "reply": "Yes — Blue Hall fits 180 with a stage...",
  "plan": { /* OperationalPlan, present once enough info gathered */ },
  "proposedActions": [{ "type": "hold_reservation", "label": "Hold Blue Hall", "payload": {…}
}],
  "requiresApproval": true }

// POST /plan — deterministic "generate the plan" for a known request
// in:  { "requestId": "req_8x2" }   // or a full EventRequestDraft
// out: OperationalPlan
```

```
// OperationalPlan — the headline artifact the agent assembles
{ "requestId": "req_8x2", "feasible": true,
  "space": { /* Space */ },
  "reservation": { /* Reservation */ },
  "quote": { /* Quote */ },
  "tasks": [ /* Task[] */ ],
  "conflicts": [],
  "alternatives": [],          // populated when feasible=false
  "narrative": "Blue Hall on 22 Jul, theater for 180, stage + 2 mics reserved. Total 134,000
ALL. No conflicts. 6 setup tasks queued." }
```

---

5. Real-time events (NATS) — the "command center" wow

Optional but high-impact and cheap. Core flow works over REST alone; treat NATS as the layer that makes the dashboard feel alive and lets the AI react proactively.

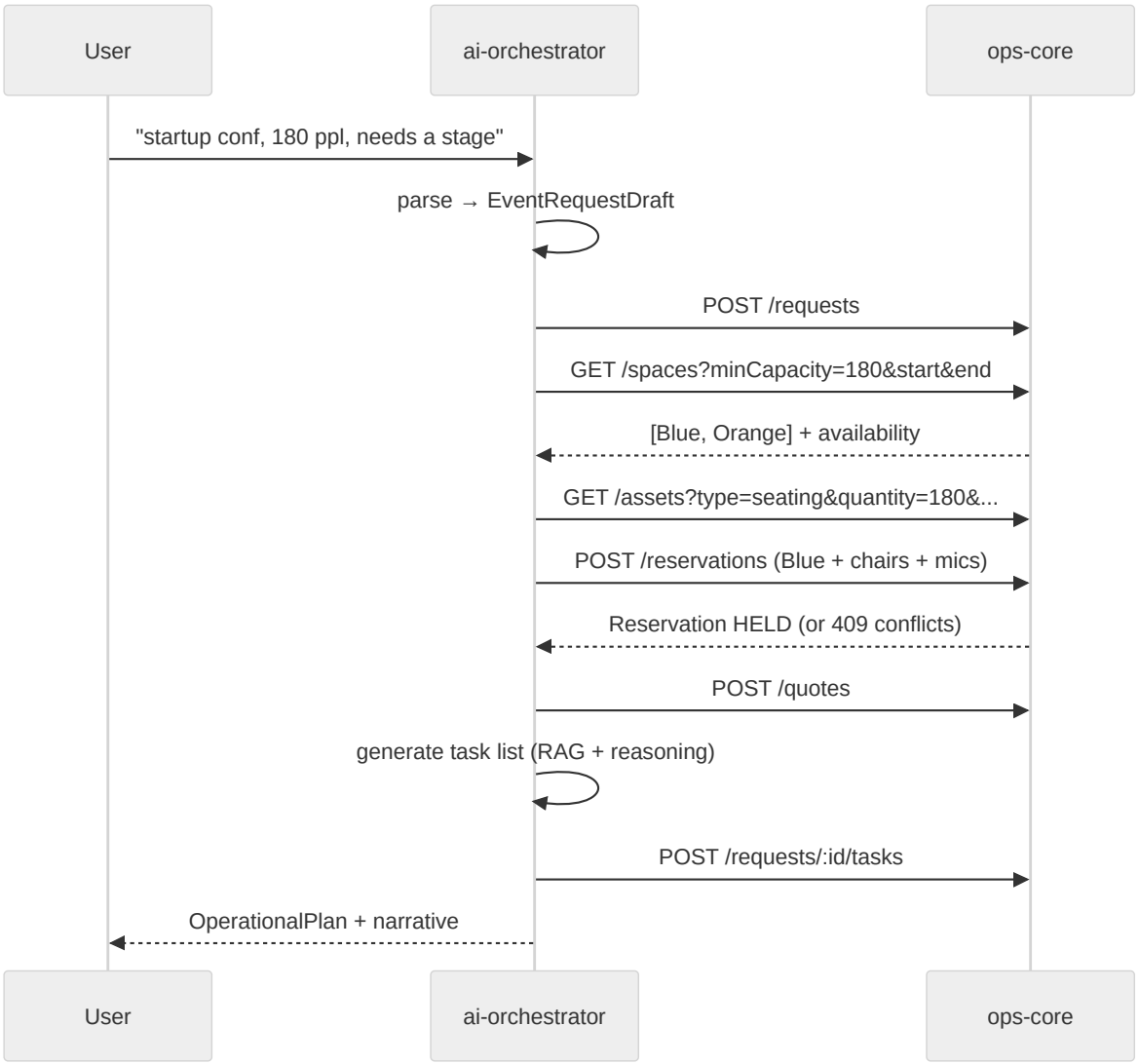
Subject	Emitted by	Consumed by	Payload
`request.created`	ops-core	UI	`EventRequest`
`reservation.held`	ops-core	UI	`Reservation`
`reservation.confirmed`	ops-core	UI	`Reservation`
`conflict.detected`	ops-core	**ai-orchestrator** + UI	`Conflict`
`request.approved`	ops-core	UI	`EventRequest`
`inventory.low`	ops-core	UI	`{ assetId, remaining }`

On `conflict.detected`, the AI can push an unprompted "heads up — this clashes with X, want me to re-plan?" That's a strong moment on stage.

---

6. Core flows

6.1 Intake → Plan (the money demo)



6.2 Conflict → alternative

Reservation returns `409 { conflicts }` → agent's conflict branch fires → queries other spaces/windows → returns `feasible:false` with `alternatives[]` and a plain-language explanation.

6.3 Approval

Human approves in the UI → `POST /requests/:id/approve` → ops-core confirms reservations, writes audit, emits `request.approved` → dashboard flips to SCHEDULED, task list goes live.

---

7. Working separately — the parallel-dev playbook

The whole point: neither of you is ever blocked on the other.

1. **Hour 0–2, together:** finalize Section 4 as an OpenAPI file in the repo. Agree IDs, enums, error shapes. This is the only meeting that must happen synchronously.
2. **Alvin builds against a mock**, not against Elis. Stand up a throwaway mock of `ops-core` (Prism from the OpenAPI spec, or a ~60-line FastAPI/Express server returning the canned payloads above). Build the entire agent, RAG, and copilot against it.
3. **Elis builds the real `ops-core`** to the same spec, in isolation, with his own seed data.
4. **Integration is a one-line switch:** flip `OPS_CORE_URL` from the mock to the real service. Schedule this as a hard checkpoint (~H24–30), not a hope.
5. **Contract changes after lock are additive only.** Need a new field? Add it; don't rename. If a breaking change is truly needed, it's a 5-minute sync, then both sides update.
6. **Shared seed data:** agree the demo dataset early (4 rooms, ~6 asset types with realistic counts, 2–3 pre-existing events that create a deliberate conflict for the demo). Elis seeds ops-core; Alvin seeds the same facts into ChromaDB.

---

8. 48-hour timeline

Window	Elis — ops-core	Alvin — ai-orchestrator
H0–2	**Contract lock** (together) + repo + Docker compose	**Contract lock** + mock ops-core up
H2–10	Schema, spaces/assets, availability engine	Intake parser + LangGraph skeleton on mock
H10–20	Reservation + quote + conflict + audit + NATS	Full planning loop, conflict branch, RAG seed, copilot
H20–24	Command Center UI v1 (chat + dashboard)	Proposal/narrative formatting, alternatives logic
H24–30	**Integration checkpoint** — real ops-core wired in, payload mismatches fixed (both)	
H30–40	Real-time dashboard via NATS, demo seed + the planted conflict	Conflict-explanation polish, copilot tone, edge cases
H40–46	Demo rehearsal, slides, buffer (both)	
H46–48	Freeze + submit (both)	

Rule: **demo-able end-to-end by H30.** Everything after is polish. If you're not integrated by H30, cut scope, don't push the checkpoint.

---

9. Demo script (maps 1:1 to "what success looks like")

1. Type the messy request → AI returns **"Yes we can"** with matched space, quote, reserved assets, **remaining inventory**, no conflict, and a setup task list. *(request submitted → auto space match → quote → asset reservation → inventory visibility → operational plan)*
2. Submit a second request that **collides** with the seeded event → `conflict.detected` → AI explains it in plain language and **proposes an alternative space/time**. *(conflict detection)*

3. **Approve** the first → status flips to SCHEDULED, reservation confirmed, **audit entry** written, "what's next" task list goes live. *(approval + complete record)*
4. Point at the live dashboard updating in real time. Close with the one line: **"This replaces the emails, the spreadsheets, and the phone calls."** *(quote the CEO's pain back to them)*

---

10. Ground rules

- **The contract (Section 4) is law.** Code disagreements lose to the OpenAPI file.
- **One coupling only:** `OPS_CORE_URL`. No shared libraries, no shared DB.
- **Currency ALL, strings i18n-ready** (cheap to do now, scores on "international-ready").
- **Favor the working core loop over breadth** — the brief says so, and judges score what runs.
- **Audit everything in ops-core** — "a complete record of decisions" is an explicit requirement and an easy win.
- `docker compose up` **brings the whole system up** — ops-core, ai-orchestrator, Postgres, Redis, NATS, ChromaDB, UI. Set this up H0–2 so integration is trivial.

---

11. Questions for the Pyramid / AADF team & data to request

At a hackathon you usually get a short mentor slot, so the list below is ordered. Ask the **top 5** first; the rest are if you have time. The goal of every question is to make the seed data realistic and the demo target their real pain — both score directly ("replaces fragmented coordination", "Albania-focused relevance").

Top 5 must-asks

1. **What's the single most time-consuming part of today's process?** Build the headline demo to kill exactly that bottleneck.
2. **What does a real request look like when it arrives** — channel (email / phone / form) and what info it contains? This defines the NL intake parser's input.
3. **Who approves and who executes an event?** The real roles/teams (venue manager, AV/technical, logistics, finance) — this drives the approval chain and task assignment.
4. **What counts as a "conflict" in practice** — just double-booking, or also setup/teardown buffers, adjacent-room noise, shared staff/equipment? This defines the conflict engine.
5. **Will judges test it hands-on, and what's their background** (technical vs. operational)? Tunes how much we lean on the AI narrative vs. the live dashboard.

Understanding the operation

- How many events run per week, and how many simultaneously? (calibrates the realism of the conflict demo)
- How far ahead are requests usually made, and how often do last-minute changes happen?
- Are there standard setup/teardown buffer times between events? Typical durations?

Spaces & assets

- Full list of bookable spaces beyond Blue/Orange/Green/Yellow — capacities per layout (theater/classroom/banquet), floors, features. Which transitional/entrance/corridor areas are actually used for events?

- Rough real inventory: how many chairs, tables, microphones, screens, projectors, stage units? Are assets shared across rooms and moved around? Where stored?
- Any space or asset with special constraints (accessibility, power limits, the auditorium)?

Pricing & quotation

- Is there a real rate card for spaces and equipment? Commercial vs. community/non-profit rates? Or is pricing handled outside this workflow entirely?
- Tax/VAT handling and currency expectations (confirms ALL + i18n approach).

Process, scope & judging

- Multi-level approvals, or single sign-off?
- Is integration with existing tools (calendar, email) expected/valued, or is a standalone prototype fine?
- Preferred UI language — Albanian, English, or bilingual?
- Any data-privacy/GDPR expectations around organizer contact data we should show we've considered?
- Confirm the brief's "favor core loop over breadth" — is depth on one flow preferred to partial coverage of all requirements?

Documents & data to request

Concrete artifacts that let us seed realistic data instead of guessing (and make the demo land as *theirs*, not generic):

- **Floor plans of floors 0 and -1** — rooms + transitional areas, with per-layout capacities.
- **Current inventory / asset register** — even a rough spreadsheet of what exists and how much.
- **3–5 real (anonymized) past event requests** — the raw input exactly as it arrives. Gold for tuning the NL parser and the demo.
- **Existing rate card / price list**, if any.
- **Any existing setup/teardown checklist or run-of-show template** — turns our task generator from guesswork into their actual process.
- **The current intake form or email template** they use today.
- **Ops-team org chart** — who does what, for approval and task-assignment logic.
- **Branding assets** (logo, colors) — for command-center UI polish and the pitch deck.
- **A sample schedule/calendar export** — shows how events are tracked now, i.e. exactly what we're replacing.